

Financial Agent 功能技術報告

本系統為一個基於 LINE Bot 的智慧理財助理，整合多項 AI 技術與外部 API，提供記帳、風險預測、信用卡查詢、金融知識問答、每日新聞、稅務試算、保險測驗等功能。

系統技術堆疊總覽

層級	技術
後端框架	Flask (Python)
前端框架	React + Vite (LINE LIFF)
資料庫	MySQL (AWS RDS) × 2、ChromaDB (向量資料庫)
主要 AI 模型	GPT-4o-mini (OpenAI)、BiGRU (PyTorch, 自訓練)
Embedding 模型	google/embedding-gemma-300m (HuggingFace)
RAG 框架	LangChain + ChromaDB
訊息平台	LINE Bot SDK v3 (linebot.v3)
ORM	SQLAlchemy
部署	AWS EC2 + AWS RDS

核心串接架構

使用者透過 LINE 傳送訊息 → Flask /callback 接收 webhook → `orchestrate()` 呼叫 **GPT-4o-mini function calling** 判斷意圖 → 依 intent 分派至對應處理模組 → 回覆使用者。

每次對話皆會：

- 從 `conversation_memory` 表載入近期記憶作為 GPT context
- 將使用者訊息與 Bot 回覆寫入記憶
- 記錄 Token 用量至 `user_token_logs` 表

功能一：記帳 (expense / income)

功能說明

使用者透過自然語言記錄支出或收入，系統自動解析金額、類別並存入資料庫。

串接流程

使用者輸入 → GPT-4o-mini 解析金額/類別/備註
→ 寫入 MySQL records 表

→ 觸發 BiGRU 風險預測 (背景 Thread)

使用技術

項目	細節
意圖判斷	GPT-4o-mini function calling
資料儲存	MySQL <code>records</code> 表 (SQLAlchemy ORM)
背景執行	Python <code>threading.Thread</code>

功能二：ML 風險預測 (BiGRU)

功能說明

每次記帳成功後，系統自動在背景執行 BiGRU 模型，預測使用者未來 7 天的消費，並計算財務風險等級 (1-5 級)。風險過高時主動 LINE Push 通知。

串接流程

```
記帳寫入成功
→ bigru_service.predict_risk_for_user()
→ 讀取使用者近 30 天記帳紀錄
→ 特徵工程 (ALIGNED_FEATURE_COLS, 10 個特徵)
→ BiGRU ensemble 推論
→ 計算風險等級
→ 寫入 risk_predictions 表
→ notification_service.check_and_notify()
→ 高風險 → LINE Push 通知
```

使用技術

項目	細節
模型架構	BiGRU with Attention (PyTorch)
訓練方式	Transfer Learning：IBM 消費資料預訓練 → 個人記帳資料 fine-tune
推論方式	Ensemble (多模型平均)
模型儲存	<code>.pkl</code> 權重檔 (<code>artifacts_bigru_tl/</code>)
特徵數量	10 個對齊特徵 (<code>ALIGNED_FEATURE_COLS</code>)
輸入序列	最近 30 天記帳紀錄
輸出	預測未來 7 天消費金額 + 風險等級 1-5
資料儲存	MySQL <code>risk_predictions</code> 、 <code>risk_notifications</code> 表

項目	細節
推播機制	LINE Bot Push Message API（有冷卻機制防重複通知）
啟動預熱	<code>app.py</code> 啟動時在背景 Thread 預載模型（ <code>_warmup_ml()</code> ）

功能三：查帳（query_expense）

功能說明

查詢使用者最近 5 筆收支紀錄，以格式化清單回覆。

串接流程

使用者詢問 → GPT-4o-mini 判斷 intent = query_expense

- 查詢 MySQL records 表 (依 line_user_id 排序)
- 格式化輸出 → LINE 回覆

使用技術

項目	細節
資料查詢	SQLAlchemy + MySQL
API	無外部 API（純 DB 查詢）

功能四：信用卡回饋查詢

功能說明

使用者輸入消費場景或商家名稱，系統查詢各銀行信用卡回饋資料庫，並由 GPT 生成比較說明回覆。

串接流程

使用者輸入

- `ai_parser.py`：GPT-4o-mini 抽出多個品牌候選名稱（含信心分數）
- `benefit_query.py`：
 - ① FTS 全文檢索 (FULLTEXT INDEX)
 - ② 無結果 → LIKE 模糊查詢 fallback
- `format_benefit_summary.py`：整理成結構化摘要 dict
- `ai_reply.py`：GPT-4o-mini 生成自然語言比較回覆
- LINE Push Message

使用技術

項目	細節
意圖 + 品牌解析	GPT-4o-mini function calling
資料庫搜尋	MySQL FULLTEXT INDEX (FTS) + LIKE fallback
卡表對應	<code>BANK_CARD_MAP</code> : table_name → (銀行, 卡名)
回覆生成	GPT-4o-mini
資料庫	MySQL <code>credit_card_benefits</code> (獨立 DB 引擎)
執行方式	背景 Thread (避免 LINE reply token 超時)

功能五：金融知識問答（RAG）

功能說明

使用者提問金融相關問題，系統從金融知識 PDF 向量資料庫中檢索相關段落，結合 GPT 生成回答。

串接流程

使用者提問

→ HuggingFace Embedding 將問題向量化

→ ChromaDB 向量搜尋 (Top-K 相關段落)

→ GPT-4o-mini 結合段落生成回答

→ LINE Push Message

離線建庫 (rebuild_chroma.py)

載入 `Financial_knowledge_final.pdf`

→ 文字切段 (Chunking)

→ Embedding 向量化

→ 存入 ChromaDB (`knowledge/chroma_db/`)

使用技術

項目	細節
Embedding 模型	<code>google/embedding-gemma-300m</code> (HuggingFace)
向量資料庫	ChromaDB
RAG 框架	LangChain (<code>langchain-huggingface</code> 、 <code>langchain-chroma</code>)
生成模型	GPT-4o-mini
知識來源	<code>knowledge/Financial_knowledge_final.pdf</code>
執行方式	背景 Thread

功能六：每日財經新聞

功能說明

使用者詢問財經新聞時，系統依使用者主題抓取最新 RSS 文章與市場行情，由 GPT 生成摘要後回覆。

串接流程

使用者輸入

- `intent_recognizer.py` : GPT 識別主題，取 `broad_query`
- `rss_fetcher.py` : 抓取 RSS 文章，依 `broad_query` 向量搜尋（限 24 小時內）
- `market_data.py` : `yfinance` 抓取今日大盤行情
- 文章不足 → `perplexity_search.py` fallback
- `openai_news.py` : GPT-4o-mini 生成新聞摘要
- 存入 MySQL `daily_news` 表（快取）
- LINE 回覆

使用技術

項目	細節
主題識別	GPT-4o-mini
新聞來源	RSS Feeds (<code>rss_config.json</code> 設定)
市場資料	<code>yfinance</code> API
搜尋 fallback	Perplexity <code>sonar</code> 模型（即時網路搜尋 LLM，回傳含 citations）
fallback 串接方式	OpenAI SDK 相容介面 (<code>base_url=https://api.perplexity.ai</code>)
摘要生成	GPT-4o-mini
快取	MySQL <code>daily_news</code> 表
Debug 資料	原始 JSON 存至 <code>raw_data/</code>

功能七：所得稅試算

功能說明

透過多輪對話收集使用者所得、扣除額等資訊，以純本地邏輯試算台灣 114 年度綜合所得稅。

串接流程

使用者開始詢問

- 建立 `_tax_sessions[line_user_id]`
- 逐題補問 (`gross_income`、`marital_status`、`dependents` 等)
- 使用者輸入 EXIT 關鍵字 → 清除 session

→ 參數齊全 → `calculate_taiwan_tax_2026()`
→ LINE 回覆試算結果

使用技術

項目	細節
Session 管理	In-memory dict <code>_tax_sessions</code> (key = line_user_id)
試算邏輯	<code>backend/tax/tax_calculator.py</code> ，純本地計算，無外部 API
稅率年度	台灣 114 年度 (2026 年申報)

功能八：保險風險測驗

功能說明

多輪問答測驗，評估使用者的風險承受度，最終給出 RR1-RR5 等級說明與建議。

串接流程

使用者開始測驗
→ `FullInsuranceQuizHandler` 初始化 session
→ 逐題發問 (`quiz_engine.build_question_message`)
→ 使用者作答，進度存於 `quiz_engine.user_sessions`
→ 使用者輸入 EXIT 關鍵字 → 清除 session
→ 全部答完 → 計算 RR Level → LINE 回覆結果

使用技術

項目	細節
測驗引擎	<code>FullInsuranceQuizHandler</code> (<code>backend/routes/quiz_handler.py</code>)
Session 管理	In-memory dict (key = line_user_id，伺服器重啟清空)
風險等級	RR1-RR5 (保守 → 積極)
外部 API	無

功能九：短期對話記憶

功能說明

每次對話的使用者訊息與 Bot 回覆都會寫入 MySQL，並在下次對話時載入作為 GPT 的短期記憶，讓系統能理解代名詞、省略的品項或連貫的對話脈絡。此外新增 `remember_context` 意圖，讓使用者主動告知背景資訊 (地點、計畫、偏好)。

串接流程

收到訊息

- `_load_recent_memory()`：載入未過期記憶 + 刪除已過期記錄 (lazy cleanup)
- 記憶作為 messages context 傳入 `orchestrate()`
- 功能處理完畢
- `_remember_message(role=user)`：寫入使用者訊息
- `_remember_message(role=assistant)`：寫入 Bot 回覆

使用技術

項目	細節
儲存	MySQL <code>conversation_memory</code> 表
TTL	2 小時 (<code>MEMORY_TTL_HOURS=2</code>)
最大筆數	10 則 (<code>MEMORY_MAX_MESSAGES=10</code>)
每則上限	500 字元 (<code>MEMORY_MAX_CHARS_PER_MESSAGE=500</code>)
清理策略	Lazy cleanup (讀取時順帶刪除過期資料)
持久化	存於 MySQL，伺服器重啟不清空

功能十：欲望清單 (wishlist)

功能說明

使用者透過 LINE 新增想買的商品至欲望清單，也可透過 LIFF 網頁介面管理清單。

串接流程

- 使用者輸入 → GPT-4o-mini 判斷 intent = wishlist
- 寫入 MySQL wishlist 表
 - LINE 回覆確認

使用技術

項目	細節
意圖判斷	GPT-4o-mini function calling
資料儲存	MySQL <code>wishlist</code> 表
防誤判機制	wishlist guard：若語意更像 remember_context 則降級處理
CRUD API	<code>/api/wishlist</code> (Blueprint)

功能十一：LIFF 前端 & Google 登入

功能說明

LINE LIFF（LINE Front-end Framework）前端提供 Google OAuth 登入、個人資料填寫、儀表板等功能，並將使用者 Google 帳號與 LINE 帳號進行綁定。

串接流程

使用者開啟 LIFF 頁面

→ Google OAuth 授權

→ Flask /auth 路由處理 callback

→ JWT 產生 token

→ 寫入 / 更新 MySQL users 表

→ LIFF 頁面顯示個人資料 / 儀表板

使用技術

項目	細節
前端框架	React + Vite
登入機制	Google OAuth 2.0 (CLIENT_ID / CLIENT_SECRET)
Token 驗證	PyJWT
部署平台	LINE LIFF + AWS EC2
帳號綁定	Google 帳號 ↔ LINE line_user_id

功能十二：Token 用量追蹤

功能說明

每次 AI 呼叫後記錄 Prompt / Completion Token 用量，累計寫入資料庫，並設定每日用量上限保護成本。

串接流程

AI 功能執行完畢

→ upsert_pipeline_tokens()

→ 累加寫入 MySQL user_token_logs 表

→ 每次收訊息時 is_over_daily_limit() 檢查

→ 超限 → 拒絕服務並提示使用者

使用技術

項目	細節
追蹤模組	backend/utils/token_tracker.py

項目	細節
儲存	MySQL <code>user_token_logs</code> 表
每日上限	50,000 tokens (<code>DAILY_TOKEN_LIMIT</code> ，可透過環境變數調整)
白名單	<code>WHITELIST_USER_IDS</code> ：開發者帳號不受限

API 與外部服務總覽

服務	用途	相關功能
OpenAI GPT-4o-mini	意圖判斷、品牌解析、回覆生成、新聞摘要、金融問答	幾乎所有功能
LINE Bot SDK v3	接收訊息、回覆、Push 通知	所有 LINE 互動
LINE LIFF	前端嵌入式網頁	登入、個人資料、儀表板
HuggingFace (google/embedding-gemma-300m)	文字向量化	金融問答 RAG
ChromaDB	向量資料庫	金融問答 RAG
yfinance	即時股市 / 大盤資料	每日新聞
Perplexity API	新聞搜尋 fallback	每日新聞
AWS RDS (MySQL)	主要資料儲存	所有功能
AWS EC2	伺服器部署	系統運行